

TALLER PROGCOMP: TRACK OPTIMIZACIÓN

INTRODUCCIÓN PROGRAMACIÓN DINÁMICA

Gabriel Carmona Tabja

Universidad Técnica Federico Santa María,
Università di Pisa

April 5, 2026

Part I

MOTIVACIÓN

MOTIVACIÓN

Caso de ejemplo: Secuencia de Fibonacci

Surgieron en la matemática india hacia el 200 a. C., en los estudios de Pingala sobre patrones de poesía sánscrita.

Fueron introducidos en Europa por Leonardo de Pisa (Fibonacci) en 1202, en su libro Liber Abaci.

$$F_n = \begin{cases} 0, & \text{si } n = 0, \\ 1, & \text{si } n = 1, \\ F_{n-1} + F_{n-2}, & \text{si } n \geq 2. \end{cases}$$

CÓDIGO RECURSIVO

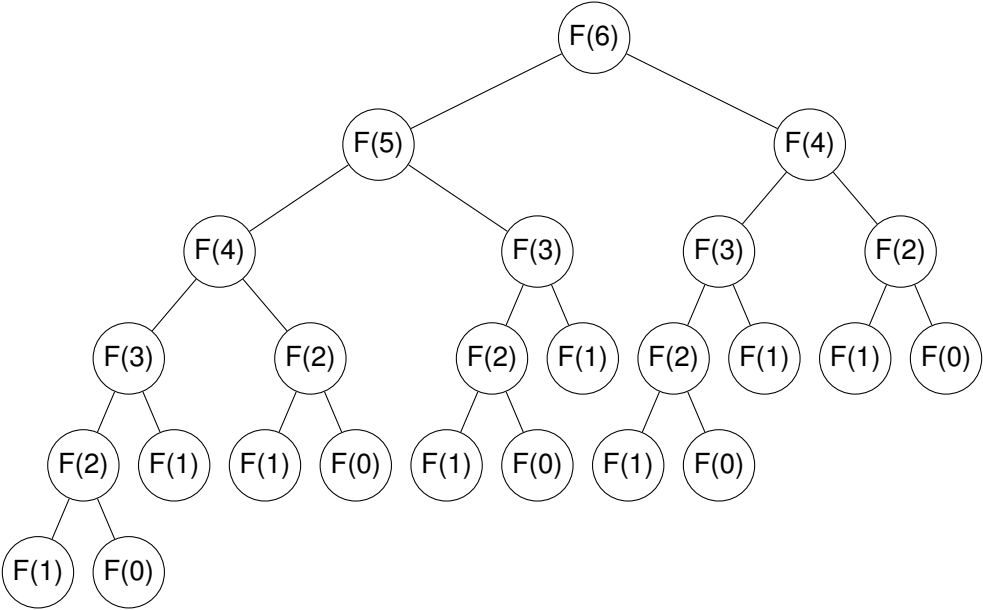
```
1 long long int rec_f(long long n) {  
2     if(n == 0) return 0;  
3     if(n == 1) return 1;  
4  
5     return rec_f(n - 1) + rec_f(n - 2);  
6 }
```

CÓDIGO RECURSIVO

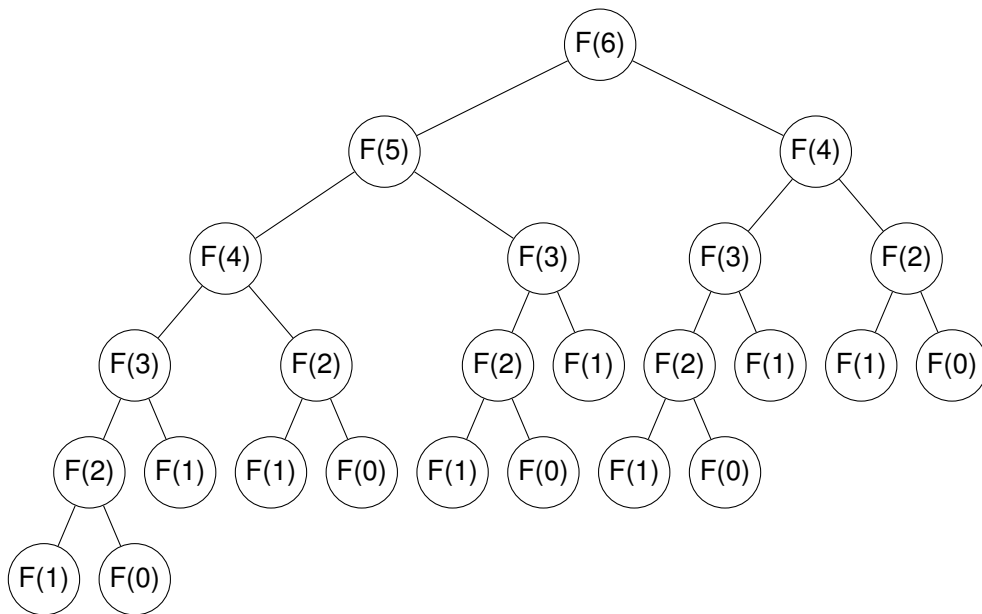
```
1 long long int rec_f(long long n) {  
2     if(n == 0) return 0;  
3     if(n == 1) return 1;  
4  
5     return rec_f(n - 1) + rec_f(n - 2);  
6 }
```

¿Cuál es la complejidad temporal de este código?

ÁRBOL DE F(6)

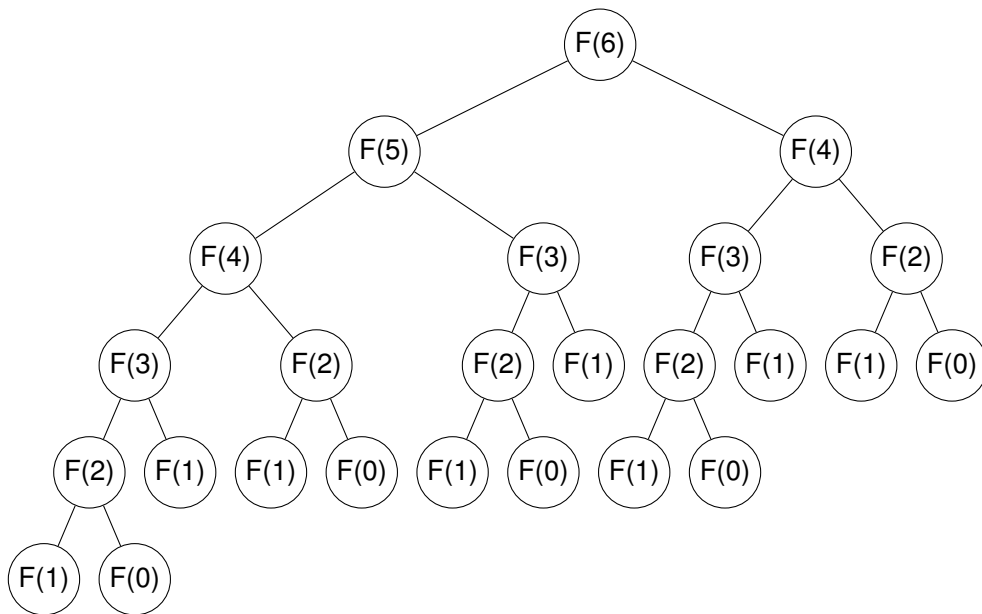


ÁRBOL DE $F(6)$



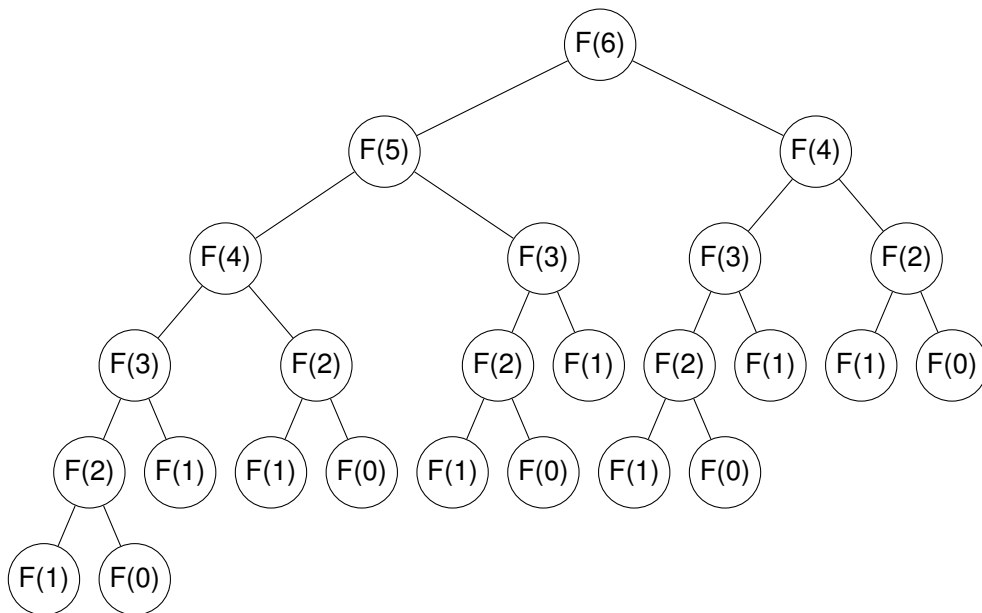
Complejidad temporal: $O(2^n)$.

ÁRBOL DE $F(6)$



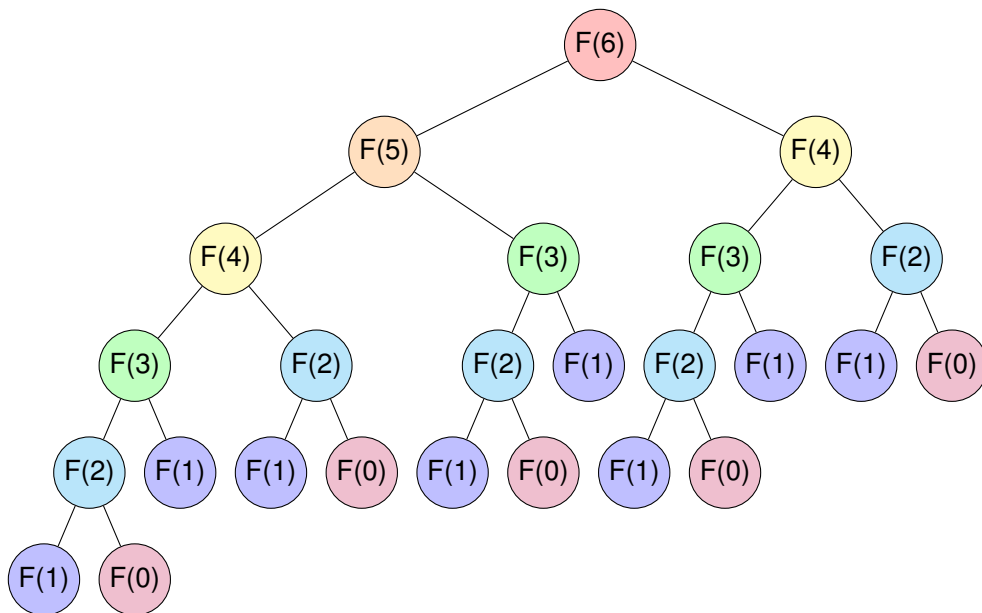
Complejidad temporal: $O(2^n)$.
¿Se podrá hacer mejor?

ÁRBOL DE $F(6)$



Complejidad temporal: $O(2^n)$.
¿Se podrá hacer mejor? Sí!

ÁRBOL DE $F(6)$



CÓDIGO MEJORADO: MEMOIZATION

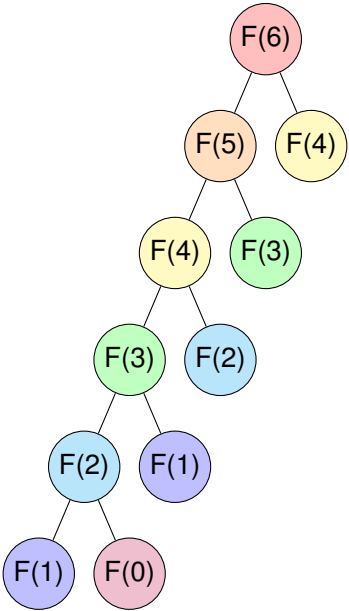
```
1  vector< long long int > memo_f;
2
3  long long int rec_f(long long n) {
4      if(memo_f[n] != -1) {
5          // ya calcule el valor por lo que puedo retornarlo directamente
6          return memo_f[n];
7      }
8      if(n == 0) {
9          memo_f[n] = 0;
10         return 0;
11     }
12     if(n == 1) {
13         memo_f[n] = 1;
14         return 1;
15     }
16
17     memo_f[n] = rec_f(n - 1) + rec_f(n - 2); // guardo la respuesta para futuras llamadas
18     return memo_f[n];
19 }
20
21 int main() {
22     int n;
23     cin >> n;
24     memo_f.resize(n + 1, -1); // -1 quiere decir que aun no hemos guardado nada
25
26     cout << rec_f(n) << "\n";
27 }
```

CÓDIGO MEJORADO: MEMOIZATION

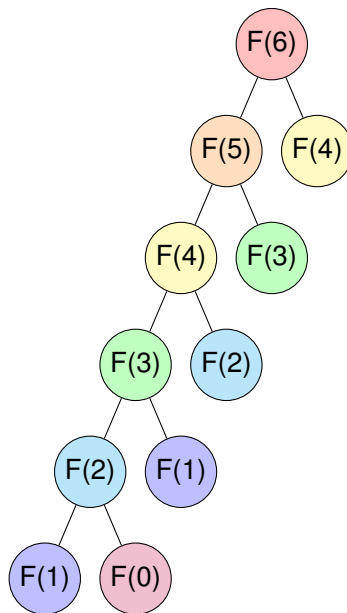
```
1  vector< long long int > memo_f;
2
3  long long int rec_f(long long n) {
4      if(memo_f[n] != -1) {
5          // ya calcule el valor por lo que puedo retornarlo directamente
6          return memo_f[n];
7      }
8      if(n == 0) {
9          memo_f[n] = 0;
10         return 0;
11     }
12     if(n == 1) {
13         memo_f[n] = 1;
14         return 1;
15     }
16
17     memo_f[n] = rec_f(n - 1) + rec_f(n - 2); // guardo la respuesta para futuras llamadas
18     return memo_f[n];
19 }
20
21 int main() {
22     int n;
23     cin >> n;
24     memo_f.resize(n + 1, -1); // -1 quiere decir que aun no hemos guardado nada
25
26     cout << rec_f(n) << "\n";
27 }
```

¿Cuál es la complejidad temporal?

ÁRBOL DE F(6) CON MEMOIZATION



ÁRBOL DE F(6) CON MEMOIZATION



Complejidad temporal: $O(n)$

Complejidad espacial: $O(n)$

CÓDIGO MEJORADO: ITERADO

```
1 long long int iter_f(long long int n) {  
2     if(n == 0) return 0;  
3     if(n == 1) return 1;  
4  
5     // ahora vamos en n = 2  
6     long long f_n_2 = 0; // f(n - 2) = f(0)  
7     long long f_n_1 = 1; // f(n - 1) = f(1)  
8  
9     for(long long int i = 2; i <= n; i++) {  
10         long long int f_n = f_n_1 + f_n_2;  
11         f_n_2 = f_n_1;  
12         f_n_1 = f_n;  
13     }  
14     return f_n_1;  
15 }
```

CÓDIGO MEJORADO: ITERADO

```
1 long long int iter_f(long long int n) {
2     if(n == 0) return 0;
3     if(n == 1) return 1;
4
5     // ahora vamos en n = 2
6     long long f_n_2 = 0; // f(n - 2) = f(0)
7     long long f_n_1 = 1; // f(n - 1) = f(1)
8
9     for(long long int i = 2; i <= n; i++) {
10         long long int f_n = f_n_1 + f_n_2;
11         f_n_2 = f_n_1;
12         f_n_1 = f_n;
13     }
14     return f_n_1;
15 }
```

Sigue siendo $O(n)$ en complejidad temporal, pero $O(1)$ en complejidad espacial.

REFERENCES I